

Property Management System

CST 2550

Krish Patel – M01039866

Aanchal Kathiriya – M01048897

Manav Patel – M01042893

Saiyam Chauhan – M01015002

Justin Khristi – M01049157

1. Introduction

This project presents the development of a Property Management System implemented as a console-based application using C#. The system is designed to manage tenants, complaints, maintenance jobs, and invoice generation in a structured and efficient manner.

Unlike a basic implementation, the system models real-world workflows where tenant complaints are processed into maintenance jobs and later integrated into billing operations. The focus of the project is on applying software engineering principles such as modular design, data structures, algorithmic thinking, and structured testing.

Report Structure:

This report is organised as follows. Section 2 discusses the system design, including data structures and algorithms used. Section 3 explains the testing approach and test cases. Section 4 provides a conclusion including limitations and reflection. Finally, references are listed.

2. System Design & Architecture

2.1 Data Structures & Justification

A Binary Search Tree (BST) was used for tenant lookup during login. The BST stores tenants using TenantId as the key and allows efficient searching.

The justification for using a BST is that it provides faster lookup compared to a linear search, especially when the number of tenants increases. Searching in a BST has an average time complexity of $O(\log n)$, whereas a linear search would require $O(n)$.

In addition, lists were used for storing collections such as maintenance jobs and invoice lines. Lists were chosen due to their simplicity and ease of iteration when calculating totals.

2.2 Algorithm Analysis

2.2.1 Invoice Generation Algorithm

FUNCTION GenerateMonthlyInvoice(tenantId, month, year):

 baseRent ← getBaseRent(flatId)

 jobs ← getMaintenanceJobs(tenantId, month, year)

 extras ← 0

 FOR each job IN jobs:

 extras ← extras + job.cost

CREATE invoice with:

baseRent

extras

RETURN invoice

Explanation:

The algorithm retrieves base rent and maintenance jobs, then iterates through each job to accumulate the total cost. The use of a loop ensures that all maintenance expenses are included in the invoice.

Time Complexity: $O(n)$, where n is the number of maintenance jobs.

2.2.2 BST Search Algorithm

FUNCTION SearchBST(root, key):

current $\leftarrow$ root

WHILE current is not null:

IF key = current.key:

RETURN current.value

ELSE IF key < current.key:

current $\leftarrow$ current.left

ELSE:

current $\leftarrow$ current.right

RETURN null

Explanation:

The BST search algorithm compares the target key with the current node and moves left or right accordingly.

Time Complexity:

- Average: $O(\log n)$
- Worst case: $O(n)$

The system follows a layered architecture consisting of four main components:

- Menu Layer – Handles user interaction through console menus
- Service Layer – Contains business logic such as invoice generation and job handling
- Repository Layer – Manages database interaction using SQL queries
- Model Layer – Represents system entities such as Tenant, Complaint, and Invoice

This separation of concerns improves maintainability, readability, and scalability of the system.

3. Key Features & Functionality

3.1 Tenant Management

Allows adding, updating, and viewing tenants. Ensures that each flat is assigned correctly and prevents duplicate occupancy.

3.2 Complaint Handling

Tenants can raise complaints categorized as plumbing, electrical, structural, or cleaning. Complaints are stored and later processed by the manager.

3.3 Invoice Generation

The system supports two types of invoices:

- Monthly Invoice: Includes base rent and maintenance costs
- Custom Invoice: Includes manually added charges

Maintenance costs are calculated using iteration over job records and accumulated into the invoice total.

4. Implementation Details

Invoice Logic

- The system retrieves base rent and maintenance jobs for a given tenant and period. A loop is used to iterate through maintenance jobs and accumulate their cost into the invoice total.

Repository Pattern

- Database operations are handled through repository classes, which execute SQL queries using `Microsoft.Data.SqlClient`.

Input Validation

- InputHelper class ensures valid data input using parsing and validation checks.

Data Structures (BST)

- A Binary Search Tree is used for tenant lookup during login, allowing efficient search based on TenantId.

5. Testing

5.1 Testing Approach

Testing was performed using MSTest to verify core system functionality.

The following areas were tested:

- Invoice calculation logic
- Input validation handling
- Binary Search Tree operations
- Workflow consistency

Unit tests were created to validate expected outputs and ensure system stability. Testing documentation was maintained throughout development.

5.2 Test Cases

Test Case	Input	Expected Output	Result
Invoice calculation	Jobs: 50 + 30	Total = 80	Pass
BST search	TenantId = 3	Correct tenant	Pass
Invalid input	Enter text instead of number	Error handled	Pass
Empty complaint	Blank description	Rejected	Pass

6. Teamwork & Project Management

The project was developed using a team-based approach with clearly defined roles.

- Leader (Krish): Invoice system, structural and cleaning workflows, system integration
- Dev 1 (Saiyam): Plumbing module and validation
- Dev 2 (Manav): Electrical module and menu integration
- Secretary (Aanchal): Documentation and validation support

- Tester (Justin): Unit testing and validation checks

GitHub was used to manage the project with separate branches and commits for each member. Documentation such as meeting minutes, standups, and testing reports were maintained to track progress.

7. Conclusion

Summary

- The project successfully demonstrates the implementation of a structured property management system. Key software engineering principles such as modular design, separation of concerns, and testing were applied throughout development.
- The system is functional, stable, and ready for submission, with all core features implemented and tested. The project also highlights the importance of teamwork and version control in software development.

Limitations

- One limitation of the system is the use of a console-based interface, which limits usability compared to modern graphical applications.
- Another limitation is that the Binary Search Tree is not self-balancing, meaning performance may degrade in worst-case scenarios.
- Additionally, validation is basic and does not cover all edge cases.

Critical Reflection

- One of the main challenges during development was managing integration between different modules such as complaints, maintenance jobs, and invoices. Initially, this led to confusion in data flow.
- Another challenge was ensuring consistent commit history and teamwork coordination, which required structured planning.

Future Improvements

In future, the system could be improved by:

- implementing a graphical user interface (GUI)
- using a balanced tree structure (e.g., AVL tree)
- improving validation and error handling
- integrating a web-based architecture